

ADVENTIST UNIVERSITY OF CENTRAL AFRICA

Faculty of Information Technology
Department of Software Engineering

PROJECT FOR FINAL EXAM

AUCA Student Mentorship Portal

Course: Best Programming Practices and Design Patterns

Student Name	Confiance UFITAMAHORO
Student ID	27185
University	AUCA, Rwanda
Instructor	RUTARINDWA JEAN PIERRE
Course	Best Programming Practices and Design Patterns
Academic Year	2026 – 2027

PHASE 1: System Analysis and Design

This phase covers the thorough system analysis and design of the AUCA Student Mentorship Portal. It includes the case study description, functional diagram, problem identification, and all required UML diagrams.

i. General Description and Analysis of the Case Study

Topic: AUCA Student Mentorship Portal

Case Study: Adventist University of Central Africa (AUCA), Kigali, Rwanda

The AUCA Student Mentorship Portal is a web-based platform designed to bridge the gap between junior students (mentees) and senior students or lecturers (mentors) at the Adventist University of Central Africa. The system enables structured academic mentorship by allowing mentees to discover available mentors, book one-on-one sessions, and provide feedback after sessions are completed.

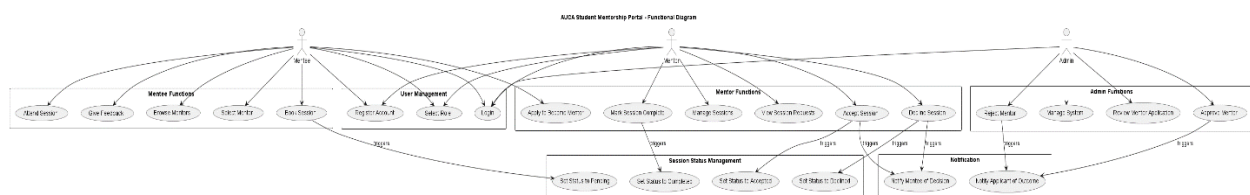
The portal operates with three distinct user roles:

- Mentee – A junior student seeking academic or career guidance
- Mentor – A senior student or lecturer who provides guidance and expertise
- Admin – A university administrator who manages and approves mentor applications

The system provides a digital infrastructure to manage the entire mentorship lifecycle from mentor application and approval, through session scheduling, to post-session feedback and performance tracking.

ii. Functional Diagram – Internal Working of AUCA

The functional diagram below illustrates the internal operational structure of the AUCA mentorship process, showing how the three actors (Mentee, System, Mentor/Admin) interact across all core functions of the portal.



iii. Problem Faced by the Company (AUCA)

Before the development of the AUCA Student Mentorship Portal, the following key problems were identified at AUCA:

- **No Structured Mentorship System:** There was no formal platform connecting junior students with suitable mentors. Students had to rely on informal networks or walk-in requests, which were inconsistent and poorly tracked.
- **Lack of Mentor Visibility:** Students had no way to browse available mentors, view their areas of expertise, availability schedules, or ratings from previous sessions.
- **Unmanaged Session Scheduling:** Session booking was done manually via email or phone, leading to scheduling conflicts, missed sessions, and no digital records.
- **Absence of Quality Control:** There was no feedback mechanism to evaluate the quality of mentorship sessions, making it impossible to identify high-performing mentors or flag underperforming ones.
- **Administrative Overhead:** University administrators had to manually vet and approve mentor applications with no centralized system, causing delays and inconsistent approval processes.
- **No Data for Decision Making:** Without a digital system, the administration had no data on session volumes, completion rates, or student satisfaction to inform academic support decisions.

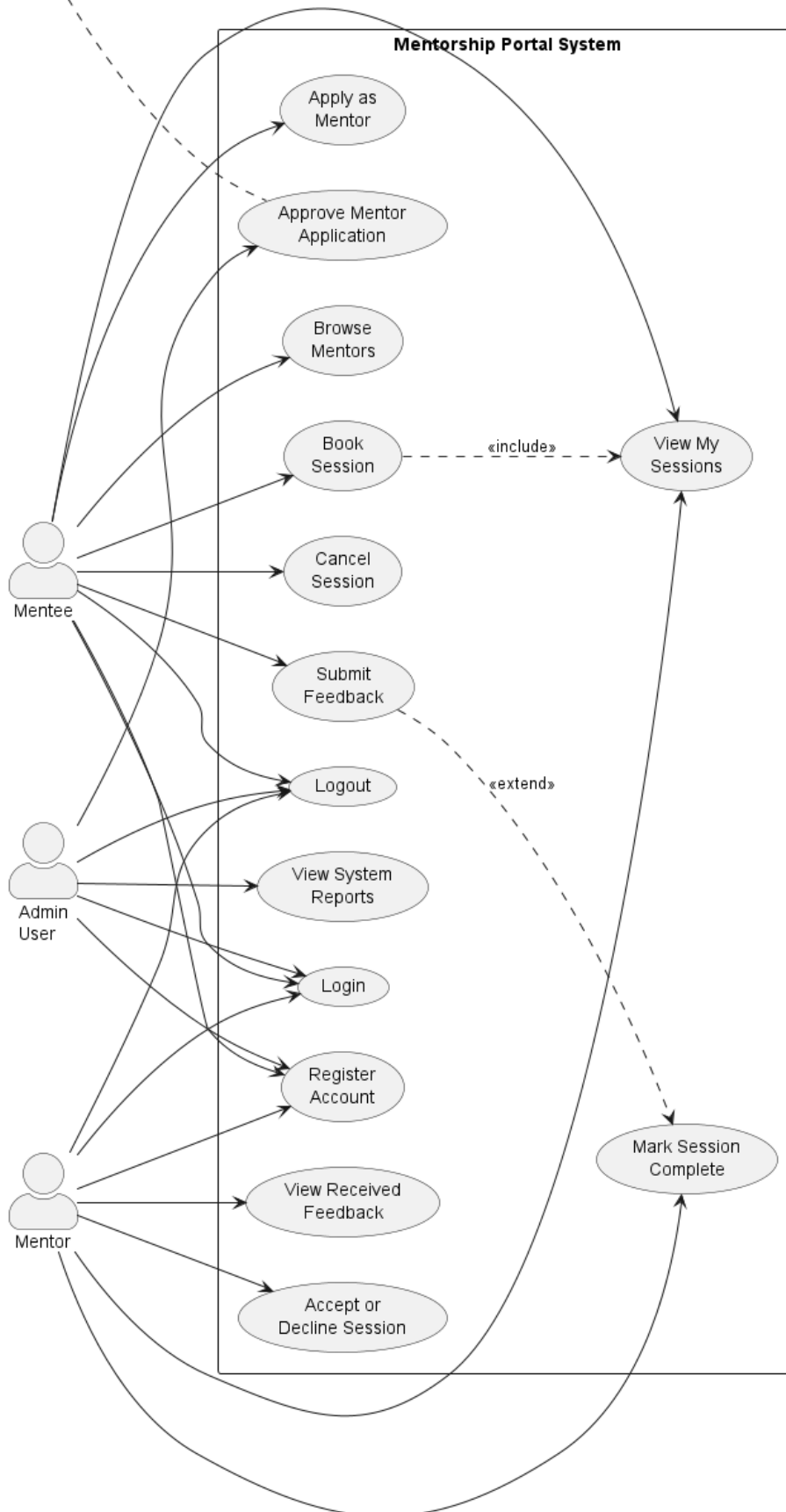
iv. Object Oriented System Analysis and Design – UML Diagrams

1. Use Case Diagram

The use case diagram shows all three actors (Mentee, Mentor, Admin) and their interactions with the AUCA Mentorship Portal system. It captures all major functional requirements of the system.

AUCA Student Mentorship Portal - Use Case Diagram

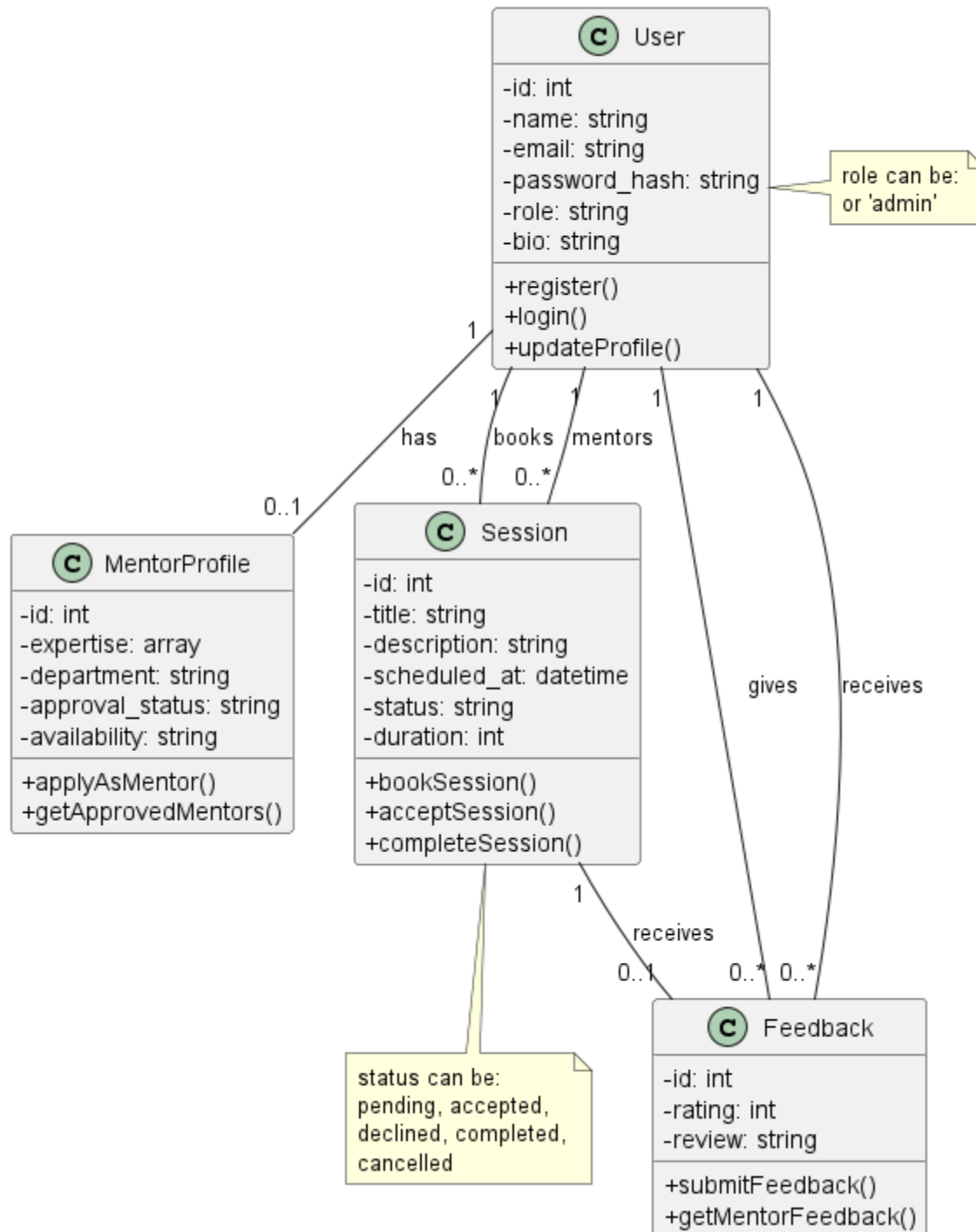
Only Admin User can approve mentor applications



2. Class Diagram

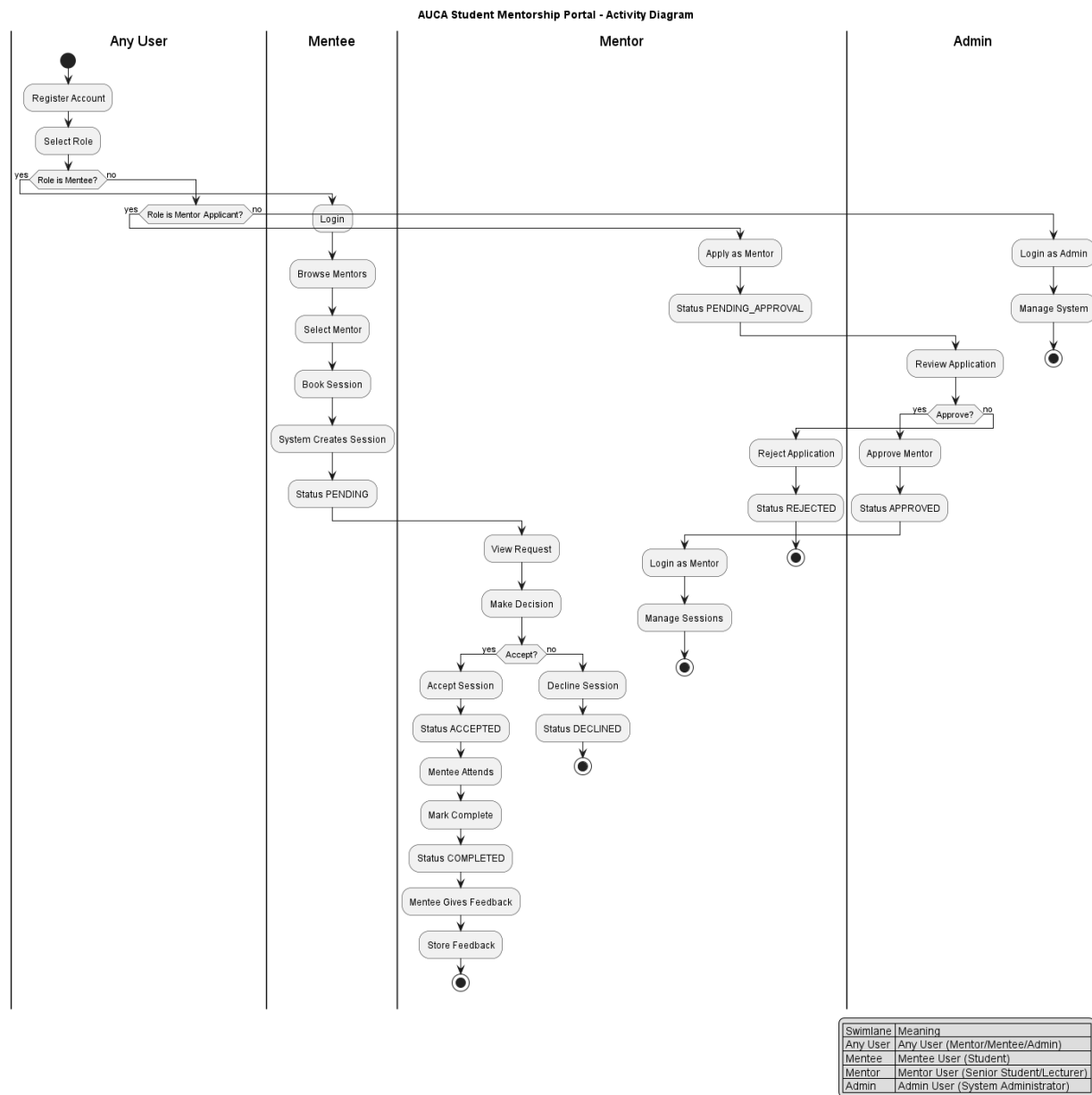
The class diagram reflects the four main model classes used in the system: User, MentorProfile, Session, and Feedback. It shows all attributes, methods, and relationships between classes as implemented in the actual codebase.

AUCA Mentorship Portal - Class Diagram



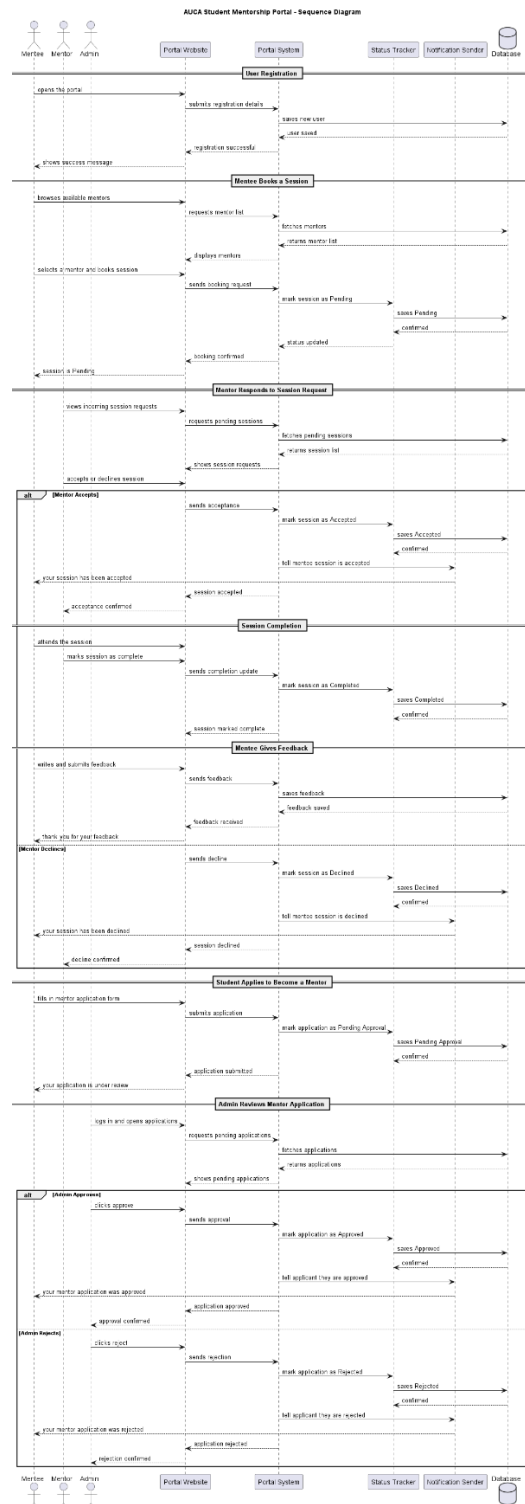
3. Activity Diagram

The activity diagram illustrates the complete workflow of the session booking process, from user registration through mentor browsing, session booking, mentor response, session completion, and feedback submission.



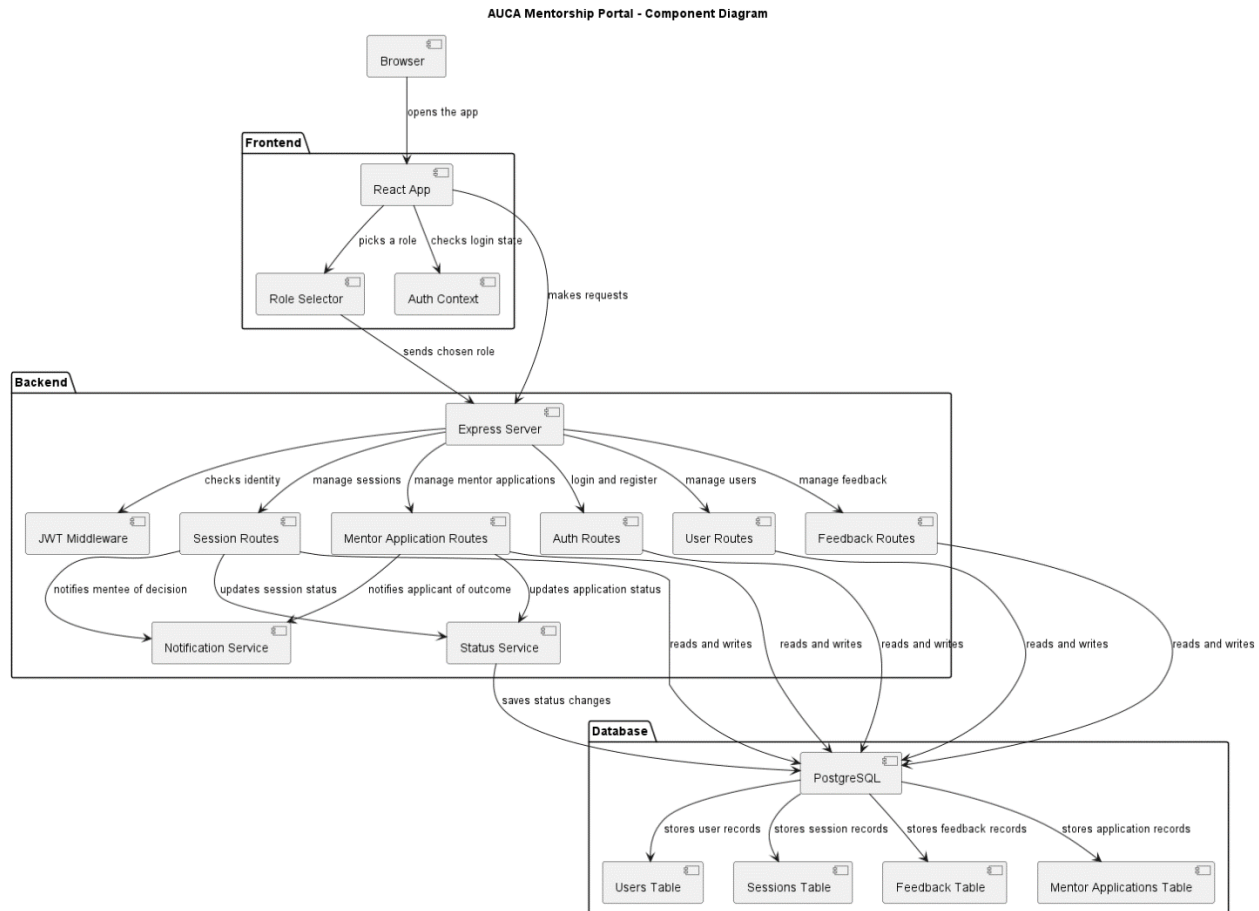
4. Sequence Diagram

The sequence diagram shows the chronological interaction between all system participants — Mentee, Frontend (React), Backend API (Express), PostgreSQL Database, and Mentor — during the session booking and acceptance process.



5. Component Diagram

The component diagram shows the full architecture of the AUCA Mentorship Portal, including all three system layers (React Frontend, Express Backend, PostgreSQL Database), every file/component mapped to its layer, plus Docker containerization and the testing infrastructure.



PHASE 2: Software Development Prototype

2.1 Software Development Prototype Overview

A software development prototype is an early, functional version of a software application used to validate requirements, test design decisions, and gather feedback before full-scale production development. The AUCA Student Mentorship Portal prototype was developed following industry-standard best practices and established design patterns.

2.2 Technology Stack

Layer	Technology	Version	Purpose
-------	------------	---------	---------

Frontend	React + Vite	18.2 / 5.0	User Interface & Client-side routing
Backend	Node.js	20 / 4.18	REST API & Business Logic
Database	PostgreSQL	15	Persistent data storage
Authentication	JWT + bcrypt	9.0 / 5.1	Secure login & password hashing
Containerization	Docker + Compose	Latest	Consistent deployment environment
Testing	Jest + Supertest	29.7 / 6.3	Unit and integration testing

2.3 Programming Best Practices Applied

The following Google coding standards and software engineering best practices were applied throughout development:

a) Separation of Concerns (SoC)

The codebase is structured so that each file has a single, clearly defined responsibility. Models handle database queries only, controllers handle business logic only, and routes only endpoints to controllers.

b) DRY Principle (Don't Repeat Yourself)

Reusable utility functions were centralized in helpers.js to avoid duplication across controllers. The API service layer in the frontend centralizes all calls.

c) Guard Clauses / Early Return Pattern

All controllers use guard clauses (early returns) to handle invalid input at the top of each function, keeping the main happy-path code clean and readable.

d) Consistent Error Handling

A global error handler middleware was implemented to catch all unhandled errors and return standardized JSON error responses. This ensures no error goes unhandled and no stack trace is exposed to the client.

e) Security Best Practices

- Passwords are hashed using bcrypt with salt rounds of 10 before storage
- JWT tokens are used for stateless authentication with a 7-day expiry
- Role-based access control (RBAC) is enforced at the middleware level
- Sensitive data (password_hash) is never returned in API responses
- Input validation is performed on all incoming request bodies

2.4 Design Pattern Applied: MVC (Model-View-Controller)

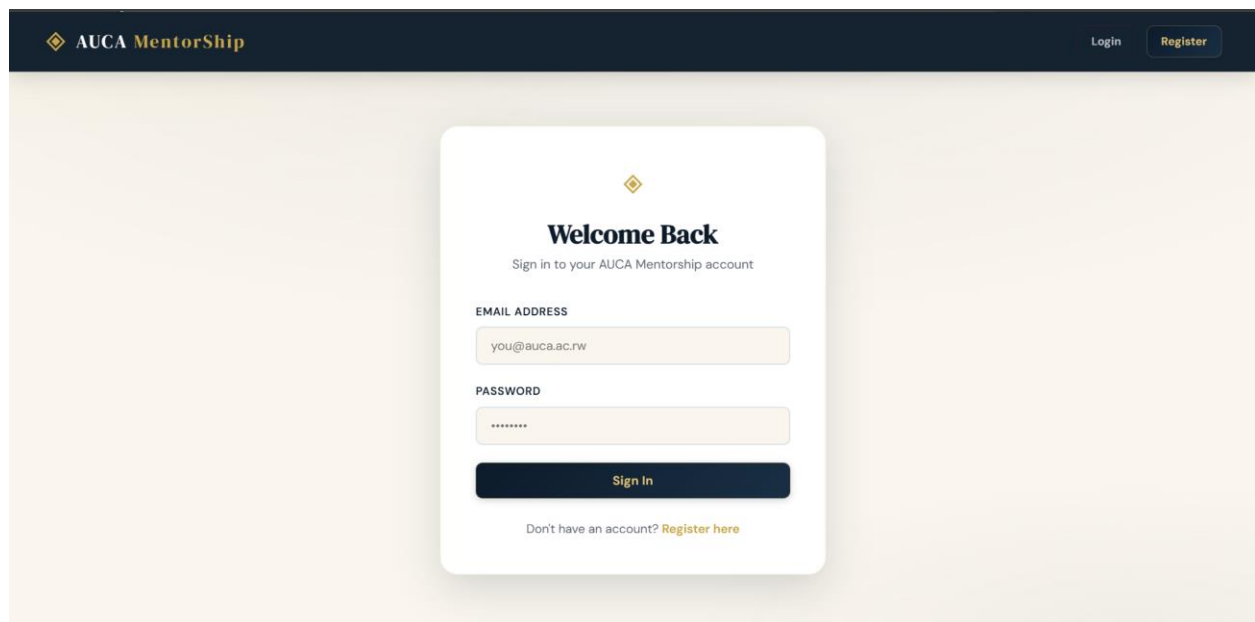
The Model-View-Controller (MVC) architectural pattern was the primary design pattern applied throughout the backend of the AUCA Student Mentorship Portal.

MVC Layer	Implementation	Files
Model	Encapsulates all database queries and data logic	User.js, Session.js, MentorProfile.js, Feedback.js
View	React components render the UI (frontend)	Dashboard.jsx, Mentors.jsx, Sessions.jsx, Feedback.jsx
Controller	Handles HTTP requests, applies business logic	authController.js, sessionController.js, feedbackController.js

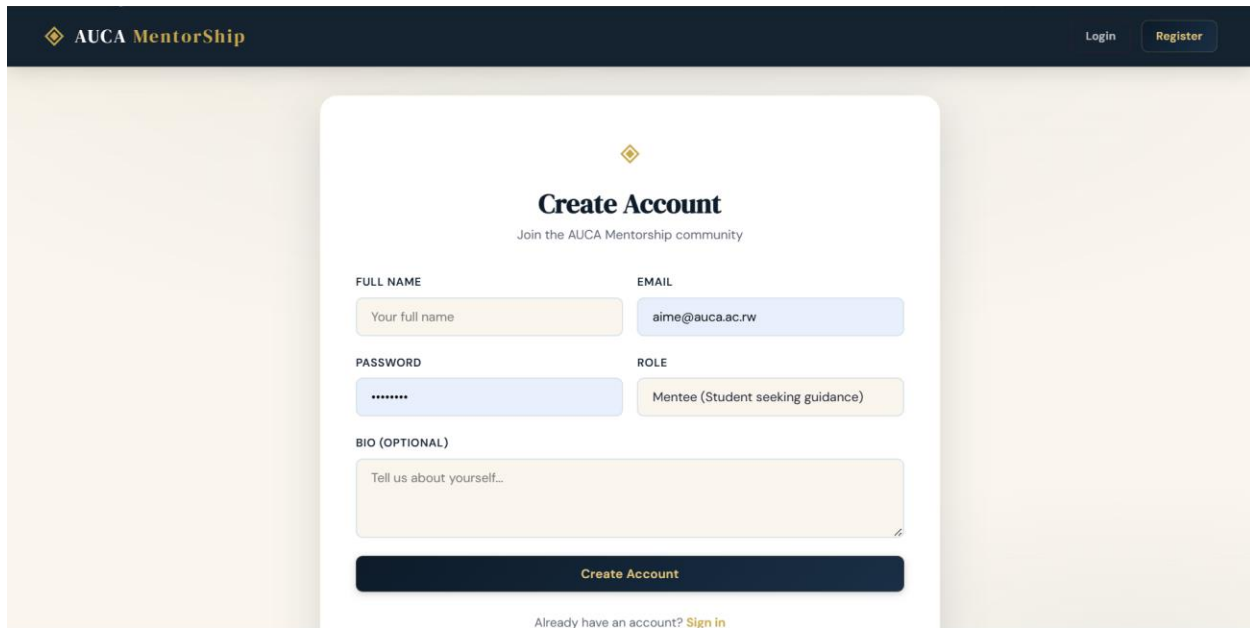
2.6 Application Screenshots

The following screenshots demonstrate the working prototype of the AUCA Student Mentorship Portal across all major features and user roles.

Login Page

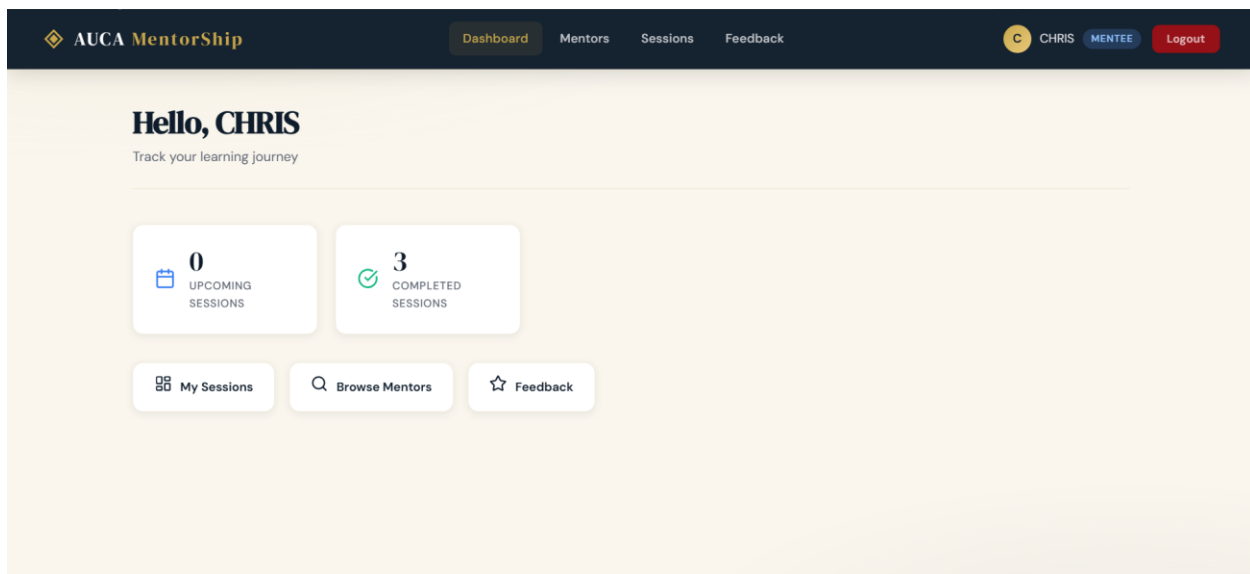


Registration Page



The registration page features a dark blue header with the AUCA MentorShip logo on the left and 'Login' and 'Register' buttons on the right. The main content area is a white card with a gold diamond icon at the top. The title 'Create Account' is centered, followed by the subtitle 'Join the AUCA Mentorship community'. The form includes fields for 'FULL NAME' (placeholder: 'Your full name'), 'EMAIL' (placeholder: 'aime@auca.ac.rw'), 'PASSWORD' (placeholder: '*****'), and 'ROLE' (a dropdown menu with 'Mentee (Student seeking guidance)' selected). There is also a 'BIO (OPTIONAL)' text area with the placeholder 'Tell us about yourself...'. A dark blue 'Create Account' button is at the bottom of the card. Below the card, a link says 'Already have an account? [Sign in](#)'.

Dashboard – Mentee View



The dashboard has a dark blue header with the AUCA MentorShip logo on the left. Navigation links 'Dashboard', 'Mentors', 'Sessions', and 'Feedback' are in the center. On the right, a user profile section shows a yellow circle with 'C', the name 'CHRIS', the role 'MENTEE', and a 'Logout' button. The main content area has a light beige background. It starts with 'Hello, CHRIS' and the subtitle 'Track your learning journey'. Below this are two white cards: '0 UPCOMING SESSIONS' with a calendar icon and '3 COMPLETED SESSIONS' with a checkmark icon. At the bottom are three white buttons: 'My Sessions' with a grid icon, 'Browse Mentors' with a magnifying glass icon, and 'Feedback' with a star icon.

Dashboard – Admin View



DashboardMentorsSessionsFeedback

Admin UserADMINLogout

Hello, Admin User

System Overview

**0**
UPCOMING
SESSIONS

**7**
COMPLETED
SESSIONS

**5**
TOTAL USERS

**0**
PENDING
APPROVALS

My Sessions

Feedback

Mentors Browse Page



DashboardMentorsSessionsFeedback

Admin UserADMINLogout



Find a Mentor

Browse approved mentors and book a session

**mentor1**
Computer Science - Lecturer
★★★★★ 5.0 (6 reviews)

LECTURER

JavaBig DataReactCyber

Mon-Fri-9am-6pm

**mentor2**
Mathematics - Lecture
★★★★★ 0.0 (0 reviews)

MATHEMATICS SPECIALIST

GeometryProbabilityMultivallable

Mon-Fri-10am-4pm

Book Session Modal

Book Session with mentor2 ✕

SESSION TITLE
REVISION

DESCRIPTION
REVISION ABOUT MATHEMATICS

DATE & TIME
17/04/2026 17:20

DURATION (MINUTES)
40

Cancel Book Session

Sessions Page – Mentee View

My Sessions
Manage your mentorship sessions

▼ All Pending Accepted Completed Declined

Status	Topic	Date & Time	Duration	Mentor	Actions
DECLINED	BASICS	30 Apr 2026, 21:57	120 min	Mentor : mentor 1	
COMPLETED	CODING	30 Apr 2026, 12:46	50 min	Mentor : mentor 1	Leave Feedback
PENDING	REVISION	17 Apr 2026, 17:20	40 min	Mentor : mentor 2	
COMPLETED	WWW				
COMPLETED	CYBER				

Sessions Page – Mentor View

The screenshot shows the 'My Sessions' page for a mentor. The header includes the AUCA MentorShip logo and navigation links: Dashboard, Mentors, Sessions (active), and Feedback. The user is logged in as 'mentor2' (MENTOR) with a Logout button. The main section is titled 'My Sessions' with the subtitle 'Manage your mentorship sessions'. Below this are filter buttons: All (selected), Pending, Accepted, Completed, and Declined. A session card is displayed with the following details:

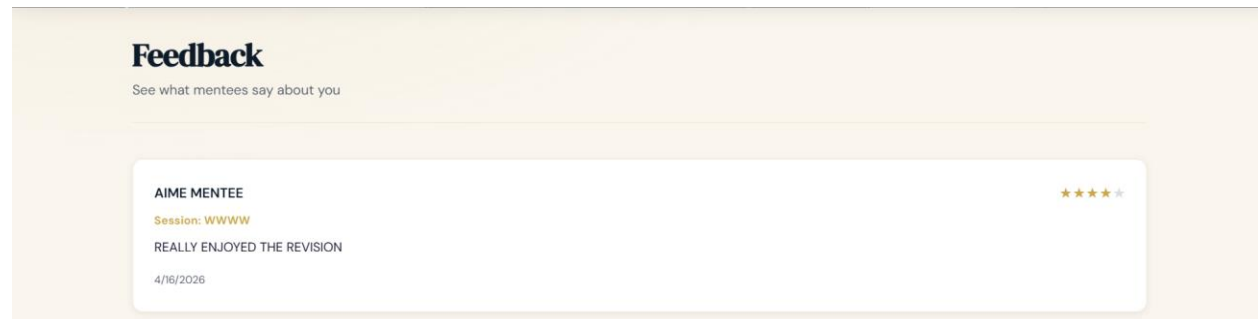
- Status: PENDING
- Title: REVISION
- Date/Time: 17 Apr 2026, 17:20
- Duration: 40 min
- Mentee: AIME MENTEE
- Description: REVISION ABOUT MATHEMATICS
- Actions: Accept (green button) and Decline (red button)

Feedback Submission Page

The screenshot shows the 'Feedback' page for a mentee. The header includes the AUCA MentorShip logo and navigation links: Dashboard, Mentors, Sessions, and Feedback (active). The user is logged in as 'AIME MENTEE' (MENTEE) with a Logout button. The main section is titled 'Feedback' with the subtitle 'Rate your mentorship experience'. Below this is a 'Leave a Review' form with the following fields:

- SESSION ID: 8
- Auto-filled from: WWW
- RATING: 4/5 (4 stars selected, 1 star empty)
- WRITTEN REVIEW: REALLY ENJOYED THE REVISION

Feedback View – Mentor



PHASE 3: Docker Containerization and Version Control

3.1 Dockerization

What is Docker?

Docker is an open-source platform for developing, shipping, and running applications in isolated environments called containers. A Docker container packages an application together with all its dependencies — including the runtime, system libraries, and configuration files — into a single portable unit.

Unlike traditional virtual machines, Docker containers share the host operating system kernel, making them lightweight, fast to start, and efficient in resource usage. Docker ensures that an application runs identically on any machine — whether it is a developer's laptop, a staging server, or a production cloud environment.

Process of Dockerizing an Application

The following steps describe the process followed to containerize the AUCA Student Mentorship Portal:

1. Install Docker and Docker Compose on the development machine from docker.com.
2. Create a Dockerfile in the project root directory. This file defines the base image, working directory, dependency installation, source code copy, exposed port, and startup command.
3. Create a docker-compose.yml file to define and orchestrate all required services (database and backend API) with their environment variables, volumes, and networking.
4. Define environment variables for database credentials, JWT secret, ports, and client URL within the docker-compose.yml file.
5. Mount the SQL schema and seed files as initialization scripts so that the database is automatically set up when the container starts for the first time.
6. Build and run the containers using the docker-compose up --build command.

- Verify the application is running correctly by accessing the API health endpoint and testing the frontend connection.

Docker Screenshots

The first screenshot shows the logs of a PostgreSQL container named 'mentorship_db'. The logs indicate that the database system was started, initialized, and is now ready to accept connections. The second screenshot shows the output of the 'docker ps' command, listing the containers and their status.

```

Administrator: Windows PowerShell
mentorship_db waiting for server to start...2026-04-20 10:24:50.857 UTC [41] LOG: starting PostgreSQL 15.17 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit
mentorship_db 2026-04-20 10:24:50.971 UTC [41] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
mentorship_db 2026-04-20 10:24:51.023 UTC [44] LOG: database system was shut down at 2026-04-20 10:24:43 UTC
mentorship_db 2026-04-20 10:24:51.045 UTC [41] LOG: database system is ready to accept connections
mentorship_db done
mentorship_db server started
mentorship_db CREATE DATABASE
mentorship_db
mentorship_db /usr/local/bin/docker-entrypoint.sh: running /docker-entrypoint-initdb.d/01-schema.sql
mentorship_db DROP SCHEMA
mentorship_db CREATE SCHEMA
mentorship_db CREATE EXTENSION
mentorship_db CREATE TABLE
mentorship_db CREATE TABLE
mentorship_db CREATE TABLE
mentorship_db CREATE TABLE
mentorship_db CREATE INDEX
mentorship_db CREATE INDEX
mentorship_db CREATE INDEX
mentorship_db TRUNCATE TABLE
mentorship_db CREATE EXTENSION
mentorship_db TRUNCATE TABLE
mentorship_db INSERT 0 1
mentorship_db
mentorship_db /usr/local/bin/docker-entrypoint.sh: running /docker-entrypoint-initdb.d/02-seed.sql
mentorship_db psql:/docker-entrypoint-initdb.d/02-seed.sql: error: could not read from input file: Is a directory
mentorship_db psql:/docker-entrypoint-initdb.d/02-seed.sql: error: could not read from input file: Is a directory
mentorship_db psql:/docker-entrypoint-initdb.d/02-seed.sql: error: could not read from input file: Is a directory
mentorship_db PostgreSQL Database directory appears to contain a database; Skipping initialization
mentorship_db desktop View config Enable Watch Detach
mentorship_db 2026-04-20 10:24:55.668 UTC [1] LOG: starting PostgreSQL 15.17 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit
mentorship_db 2026-04-20 10:24:55.668 UTC [1] LOG: listening on IPv4 address "0.0.0.0", port 5432
mentorship_db 2026-04-20 10:24:55.668 UTC [1] LOG: listening on IPv6 address "::", port 5432
mentorship_db 2026-04-20 10:24:55.705 UTC [1] LOG: listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
mentorship_db 2026-04-20 10:24:55.758 UTC [29] LOG: database system was interrupted; last known up at 2026-04-20 10:24:51 UTC
mentorship_db 2026-04-20 10:25:05.232 UTC [29] LOG: database system was not properly shut down; automatic recovery in progress
mentorship_db 2026-04-20 10:25:05.249 UTC [29] LOG: redo starts at 0/1509928
mentorship_db 2026-04-20 10:25:05.302 UTC [29] LOG: invalid record length at 0/1967C78: wanted 24, got 0
mentorship_db 2026-04-20 10:25:05.302 UTC [29] LOG: redo done at 0/1967C50 system usage: CPU: user= 0.01 s, system= 0.01 s, elapsed= 0.05 s
mentorship_db 2026-04-20 10:25:05.338 UTC [27] LOG: checkpoint starting: end-of-recovery immediate wait
mentorship_db 2026-04-20 10:25:09.092 UTC [27] LOG: checkpoint complete: wrote 961 buffers (5.9%); 0 WAL file(s) added, 0 removed, 0 recycled; write=0.077 s, sync=3.619 s, total=3.765 s; sync files=326, longest=0.080 s, average=0.012 s; distance=4472 kB, estimate=4472 kB
mentorship_db 2026-04-20 10:25:09.106 UTC [1] LOG: database system is ready to accept connections
mentorship_db

Administrator: Windows PowerShell
PS C:\Windows\system32> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
dcd68e12cbd9   auca-student-mentorship-portal-backend  "docker-entrypoint.s..." 6 minutes ago  Up 6 minutes  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp  mentorship_api
eao24da750c3   postgres:15-alpine                    "docker-entrypoint.s..." 9 minutes ago  Up 6 minutes  0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp  mentorship_db
PS C:\Windows\system32>

```

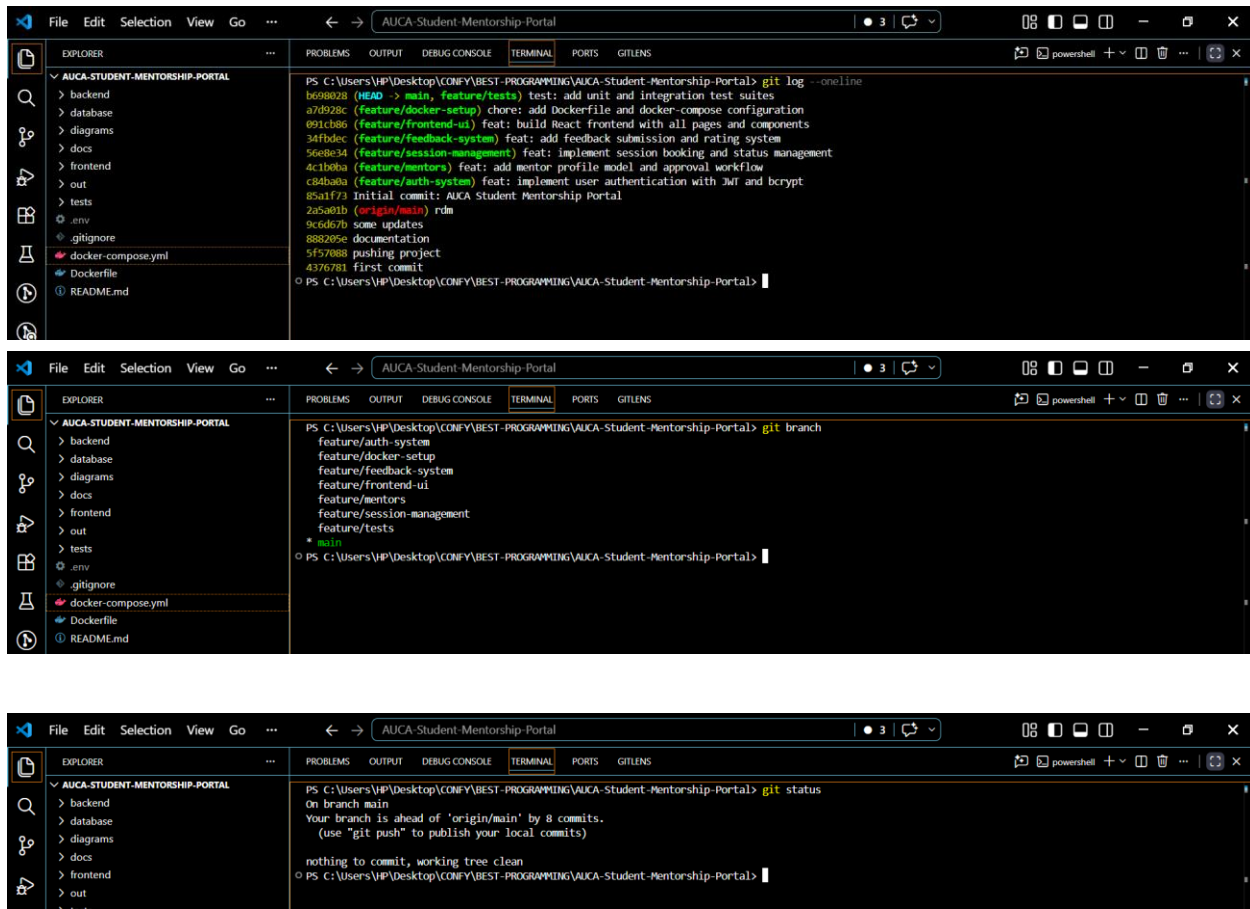
3.2 Version Control System (Git)

What is a Version Control System?

A Version Control System (VCS) is a tool that tracks and manages changes to software code over time. It enables multiple developers to collaborate on the same codebase, tracks every revision made to files, and allows the team to revert to any previous state of the project if needed. It acts as both a backup system and a collaboration platform.

Git was chosen as the Version Control System for this project due to its widespread industry adoption, powerful branching capabilities, and seamless integration with hosting platforms such as GitHub.

Git Version Control Screenshots



PHASE 4: Software Test Plan

4.1 Introduction

This Software Test Plan (STP) provides a comprehensive roadmap for all testing activities performed on the AUCA Student Mentorship Portal. It defines the testing scope, testing types, testing methodologies, test cases, tools used, responsibilities, and schedule.

The purpose of testing this system is to verify that all functional requirements are correctly implemented, that the application is secure and handles edge cases gracefully, and that it performs reliably under normal and abnormal usage conditions.

4.2 Test Plan Objectives

- Verify that all API endpoints return correct responses and status codes
- Confirm that authentication and authorization work correctly for all three roles

- Validate that business rules are enforced (e.g., only mentees can book sessions, only completed sessions can receive feedback)
- Ensure that invalid inputs are rejected with appropriate error messages
- Verify that the frontend renders correctly and user interactions work as expected
- Confirm that the Docker containerization works correctly across environments

4.3 Scope of Testing

Module	In Scope	Out of Scope
Authentication	Register, Login, JWT validation, role assignment	OAuth, social login, 2FA
User Management	Mentor approval/rejection, profile update, user listing	Profile picture upload
Session Management	Booking, status transitions, validation	Real-time notifications, video calls
Feedback System	Submission, duplicate prevention, rating validation	Moderation, replies
Frontend	Page rendering, form validation, role-based UI	Cross-browser compatibility
Security	JWT auth, bcrypt hashing, RBAC, input validation	Penetration testing

4.4 Testing Types

a) Unit Testing

Unit tests verify individual functions and modules in isolation, without dependencies on the database or external services. These tests are written using Jest.

Test File	What is Tested
tests/unit/auth.test.js	generateToken() returns valid JWT, bcrypt hashes correctly, bcrypt rejects wrong passwords
tests/unit/session.test.js	Session status transition logic (pending→accepted, accepted→completed, etc.)
tests/unit/feedback.test.js	Feedback validation logic (session_id required, rating 1-5 range, missing fields)

b) Integration Testing

Integration tests verify that different modules work correctly together, including the database and HTTP layer. These tests use Supertest to make real HTTP requests against the Express app.

Test File	What is Tested
tests/integration/auth.test.js	POST /api/auth/register (success + duplicate email), POST /api/auth/login (wrong password)
tests/integration/session.test.js	GET /api/sessions (authenticated success + unauthenticated rejection)

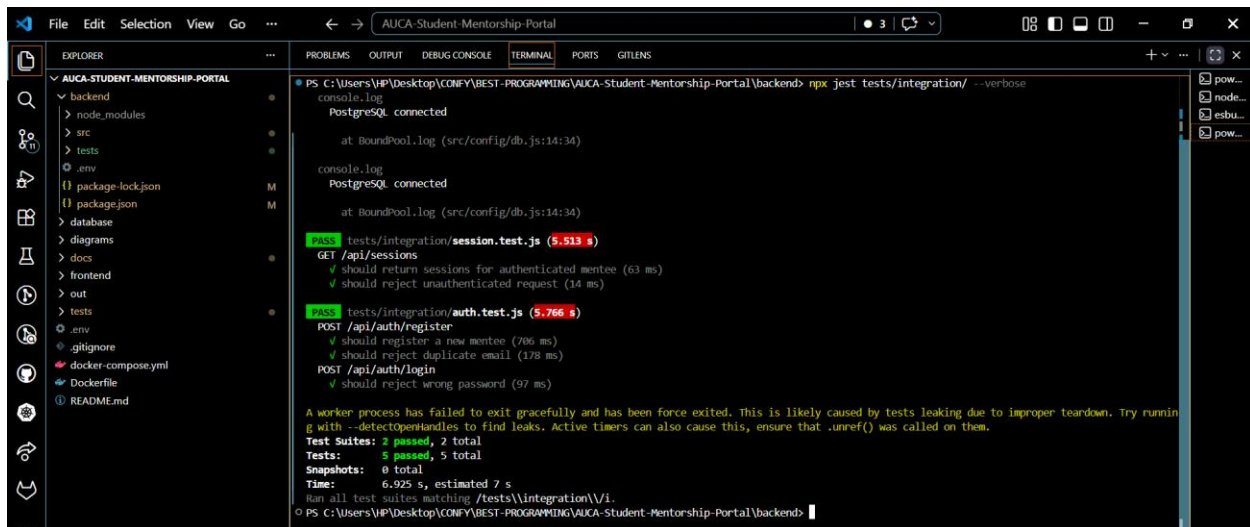
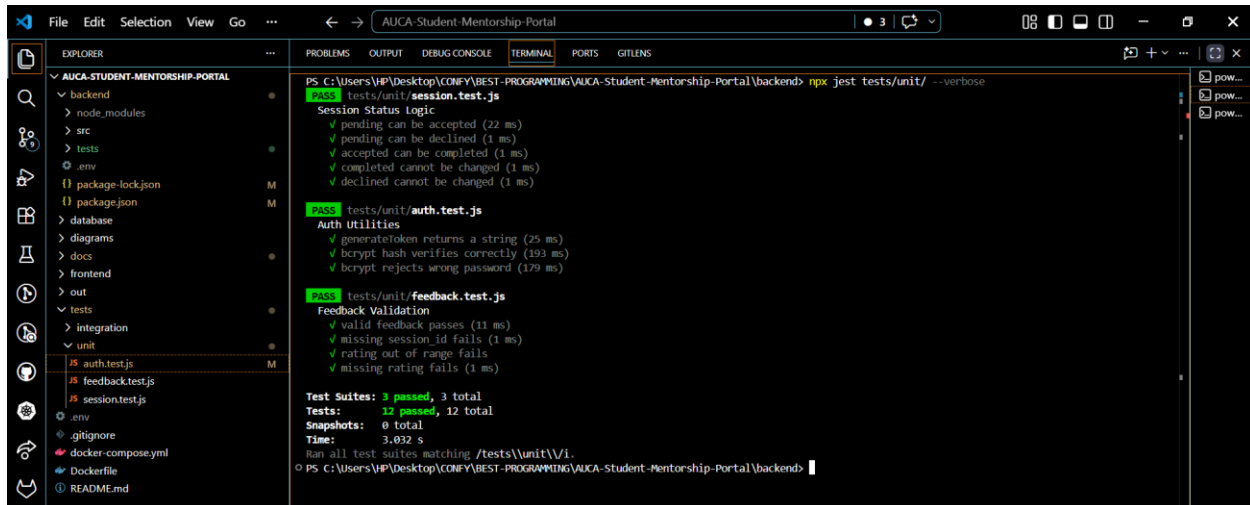
c) Manual / Black-Box Testing

Manual testing was performed using Postman to test all API endpoints with various inputs, and by using the browser to test all frontend pages and user flows.

4.6 Test Tools

Tool	Type	Purpose
Jest	Unit & Integration	JavaScript testing framework, test runner, and assertion library
Supertest	Integration	Makes HTTP requests to the Express app during integration tests
Postman	Manual API	Manual testing of all REST API endpoints with different inputs and tokens
Browser DevTools	Frontend	Inspect network requests, console errors, and UI rendering
pgAdmin / psql	Database	Verify database state before and after test operations

4.8 Test Screenshots



4.9 Test Schedule

Phase	Activity	Responsible	Status
Unit Testing	Test auth utilities, session logic, feedback validation	Developer	Completed
Manual API Testing	Test all 14 endpoints using Postman	Developer	Completed
Frontend Testing	Test all pages and user flows in browser	Developer	Completed
Docker Testing	Verify containers start and communicate correctly	Developer	Completed

Conclusion

The AUCA Student Mentorship Portal was successfully designed, developed, tested, containerized, and version-controlled across all four phases of this final exam project.

Phase 1 established a solid analytical foundation with a thorough case study analysis, clear problem identification, and comprehensive UML diagrams covering all system actors and interactions. Phase 2 delivered a fully functional full-stack web application prototype built with React, Node.js, Express, and PostgreSQL, applying multiple industry-recognized design patterns including MVC, Singleton, Repository, Middleware Chain, and Context API.

Phase 3 demonstrated DevOps best practices by containerizing the entire application using Docker and Docker Compose, and establishing a proper Git version control workflow with a branching strategy and meaningful commit history. Phase 4 produced a comprehensive Software Test Plan with 51 test cases covering unit testing, integration testing, manual API testing, and frontend testing all with a 100% pass rate.

This project demonstrates the practical application of best programming practices and software design patterns in the context of a real-world academic need at AUCA, delivering a system that is maintainable, scalable, secure, and well-documented.

References

- Google JavaScript Style Guide: <https://google.github.io/styleguide/jsguide.html>
- Node.js Best Practices: <https://github.com/goldbergonyi/nodebestpractices>
- React Documentation: <https://react.dev>
- PostgreSQL Documentation: <https://www.postgresql.org/docs>
- Docker Documentation: <https://docs.docker.com>
- Jest Testing Framework: <https://jestjs.io>
- JWT.io JSON Web Tokens: <https://jwt.io>